FITBUDDY

AI Fitness Plan Generator using Gemini Models

Project Documentation

Team ID	SWTID-2026-6912
Team Members	Pavithra M, Manisha S, Monica G, Monisha Anandan, Mridula M
Technology	FastAPI, Google Gemini, SQLite, SQLAlchemy, Jinja2, HTML/CSS

Prepared based on the supplied FitBuddy project material.

1. Project Description

FitBuddy is a web-based application that uses AI to generate personalized workout plans and nutrition tips based on a user's fitness goals, such as weight loss, muscle gain, or general wellness. The application is built with FastAPI and integrated with Google's Gemini AI models to simplify the creation of structured and goal-oriented fitness routines.

The application accepts user information such as name, age, weight, fitness goal, and preferred workout intensity. It generates a customized 7-day fitness plan and allows users to submit feedback so the plan can be updated using AI. A complementary nutrition or recovery tip is also generated according to the selected goal.

FitBuddy uses a clean HTML/Jinja2 interface and SQLite database support through SQLAlchemy. User information and generated plans are stored so that the application can retrieve original and updated plans. The system also provides API endpoints and an administrative view of registered users and their plans.

Key Features

- Personalized 7-day workout plan generation.
- Nutrition or recovery tip generation.
- Feedback-based workout plan updating.
- SQLite/SQLAlchemy storage of users and plans.
- Administrative view of users and original/updated plans.
- FastAPI backend with Jinja2-rendered HTML pages.

2. Scenarios

Scenario 1: A user enters personal details, fitness goal, and workout intensity and receives a personalized 7-day workout schedule.

Scenario 2: A user submits feedback such as requesting more cardio or additional rest days, and the system generates an updated plan.

Scenario 3: A user requests a nutrition or recovery tip based on the selected fitness goal.

Scenario 4: An administrator or coach accesses the platform to view registered users and their original and updated workout plans.

3. Technical Architecture

FitBuddy follows a modular FastAPI-based architecture. The frontend collects user information and displays generated results, the backend handles routing and application logic, Gemini provides AI generation, and SQLite with SQLAlchemy provides persistent storage.

Layer	Technology	Responsibility
Frontend	HTML + Jinja2 + CSS	User input, workout result display, feedback form, admin view
Backend	FastAPI + Python	Routing, validation, request processing, AI orchestration
AI Layer	Google Gemini models	Workout generation, plan updates, nutrition/recovery tips
Database	SQLite + SQLAlchemy	Store users, original plans, and updated plans

Application Flow

User	Jinja2 / HTML	FastAPI Routes	Gemini AI	SQLite / SQLAlchemy
Inputs + feedback	Forms + results	/generate-workout /submit-feedback /view-all-users	Workout + tip + update	Persistent records

Database note: Because FitBuddy uses SQLite with SQLAlchemy, the database structure should be represented with an ER diagram in a complete academic submission. The supplied FitBuddy material identifies user and plan storage but does not provide a formal ER diagram, so the following is a logical representation derived from those fields.

USER		PLAN
user_id (ID)	1 ——————<	plan_id (ID)
name		user_id (FK)
age		original_plan
weight		updated_plan
goal		feedback / update data
intensity		

4. Pre-requisites

- FastAPI framework knowledge and familiarity with API routing.
- Gemini API familiarity for generative AI integration.
- HTML, CSS, and Jinja2 template skills.
- Python programming proficiency.
- Version control with Git.
- SQLAlchemy and SQLite basics.
- Environment setup using pip and a virtual environment.
- Uvicorn ASGI server knowledge.

5. Project Workflow

Milestone	Activities
1. Model Selection and Architecture	1.1 Research/select Gemini model; 1.2 Define architecture; 1.3 Set up development environme
2. Core Functionalities Development	2.1 Develop workout, nutrition, feedback and storage functionality; 2.2 Implement FastAPI bac
3. routes.py Development	3.1 Write main application routing and integration logic
4. Frontend Development	4.1 Design user interface; 4.2 Create dynamic Jinja2 templates
5. Deployment	5.1 Prepare local deployment; 5.2 Test and verify local deployment

6. Milestone 1: Model Selection and Architecture

This milestone focuses on selecting the appropriate Gemini models for FitBuddy and defining how the frontend, backend, AI layer, and database interact.

Activity 1.1: Research and Select the Appropriate Generative AI Model

- Understand the need to generate structured 7-day workout plans.
- Provide concise and actionable nutrition or recovery tips.
- Update existing plans based on user feedback.
- Support real-time, web-based interaction.
- Use Gemini models because they are API-based and integrate with the FastAPI application.

Model Usage

Model	Purpose
Gemini 1.5 Pro	Structured 7-day workout generation and feedback-based plan updating.
Gemini Flash	Fast nutrition or recovery tip generation.

Activity 1.2: Define the Architecture

- Frontend: index.html for user input, result.html for plan/tip/feedback, and all_users.html for the admin panel.
- Backend: FastAPI handles form data, routes, Gemini calls, storage, and template rendering.
- AI integration: Gemini Pro handles workout generation and updates; Gemini Flash handles nutrition tips.
- Database: SQLite with SQLAlchemy stores user information and generated plans.

Activity 1.3: Development Environment

```
python -m venv fitbuddy-env
pip install fastapi uvicorn jinja2 sqlalchemy python-multipart google-generativeai
uvicorn app.main:app --reload
```

7. Milestone 2: Core Functionalities Development

Activity 2.1: Develop the Core Functionalities

Workout Plan Generation

- Function: generate_workout_gemini().
- Uses user goal and intensity to generate a structured 7-day plan.
- The generated plan contains warm-up, main workout, exercise details, sets/reps, and cooldown or recovery guidance.
- The result is returned to routes.py and displayed in result.html.

Nutrition Tip Generation

- Function: generate_nutrition_tip_with_flash().
- Generates a concise nutrition or recovery tip tailored to the selected goal.
- The tip is displayed beneath the workout plan.

Feedback-Based Plan Updating

- Function: update_workout_plan().
- Accepts user feedback such as adding yoga or increasing cardio.
- Passes the original plan and feedback to Gemini Pro and stores the revised plan.

User and Plan Storage

- Functions include save_user(), save_plan(), update_plan(), get_original_plan(), and get_user().
- SQLite records user information and generated plans through SQLAlchemy ORM.
- Original and updated plans are preserved as separate data.

Activity 2.2: FastAPI Backend

- Define routes in routes.py.
- Collect user input from index.html using FastAPI Form(...) parameters.
- Use Pydantic schemas such as UserInput and FeedbackRequest to validate request structure.
- Connect /generate-workout and /submit-feedback routes to the Gemini-powered functions.
- Return AI-generated content to the Jinja2 templates.

8. Milestone 3: routes.py Development

The routes.py layer acts as the operational bridge between the frontend templates, Gemini-powered generation modules, and the SQLite database.

Route	Function
/	Displays the homepage using index.html.
/generate-workout	Receives user input, generates workout and nutrition content, stores the data, and renders result.html.
/submit-feedback	Retrieves the original plan, sends feedback to the update function, saves the revised plan, and renders the
/view-all-users	Retrieves users and plans and displays them in the administrative view.

Gemini Integration Flow

- Workout generation sends fitness goal and intensity to Gemini Pro.
- Nutrition generation sends the selected goal to Gemini Flash.
- Feedback updates send the original plan plus user feedback to Gemini Pro.
- Responses are returned to result.html using template context and formatted blocks.

9. Milestone 4: Frontend Development

The frontend provides a user-friendly interface for entering fitness information, viewing the generated plan, submitting feedback, and allowing administrators to review stored users and plans.

Activity 4.1: Designing and Developing the User Interface

- index.html provides fields for username, user ID, age, weight, fitness goal, and workout intensity.
- The interface uses a fitness-oriented visual design with a gym-themed background and clear form controls.
- Responsive layout techniques such as Flexbox and media queries support mobile screens.
- result.html displays the generated 7-day workout plan and nutrition tip and provides a feedback form.
- all_users.html provides an administrative view of registered users and original/updated plans.

Activity 4.2: Dynamic Templates with Jinja2

- FastAPI's Jinja2Templates is used to render HTML templates dynamically.
- User information is bound to the result page through template variables.
- workout_plan and nutrition_tip are inserted into result.html.
- Feedback is submitted to /submit-feedback through a POST form.

10. Milestone 5: Deployment

The deployment milestone focuses on running FitBuddy locally with FastAPI and Uvicorn, verifying the application, and ensuring environment dependencies are correctly configured.

Activity 5.1: Preparing the Application for Local Deployment

- Create and activate a virtual environment.
- Install FastAPI, Uvicorn, SQLAlchemy, Jinja2, python-multipart, and the Google Generative AI library.
- Configure the Gemini API key using an environment variable or .env file.
- Keep sensitive API keys out of source control.

Example Environment Variable

```
GOOGLE_API_KEY=your_gemini_api_key_here
```

Activity 5.2: Testing and Verifying Local Deployment

- Start the FastAPI development server using Uvicorn.
- Open the application at http://127.0.0.1:8000.
- Use /docs to inspect and test available API endpoints.
- Test workout generation, nutrition tips, feedback-based updates, and stored records.
- Verify that both original and updated plans are preserved correctly.

11. Team Member Work Allocation

S.No	Team Member	Assigned Work
1	Pavithra M	Gemini AI integration, model selection, workout generation and AI prompt-related work.
2	Manisha S	FastAPI backend, route handling, request processing and API integration.
3	Monica G	SQLite/SQLAlchemy database, user storage, plan storage and update handling.
4	Monisha Anandan	Frontend UI, HTML/CSS, Jinja2 templates and result/feedback pages.
5	Mridula M	Deployment setup, environment configuration, testing, verification and documentation/demo suppor

12. Website Pages and User Flow

Page	Purpose
Home / index.html	Collects user details and fitness preferences.
Result / result.html	Shows the personalized 7-day workout plan, nutrition tip and feedback option.
Feedback	Accepts user feedback and triggers plan revision.
Admin / all_users.html	Shows registered users and original/updated plans.

13. Testing and Verification

- Verify required user fields are accepted and passed to the backend.
- Verify the workout generator produces a 7-day structured plan.
- Verify nutrition/recovery tips are generated for the selected goal.
- Verify feedback modifies the existing plan rather than replacing the original record.
- Verify SQLite/SQLAlchemy persistence for users and plans.
- Verify all major pages render correctly through Jinja2.
- Verify the local FastAPI server and /docs endpoint are accessible.

14. Conclusion

FitBuddy is a comprehensive AI-powered fitness planning application that combines FastAPI, Google Gemini models, SQLite/SQLAlchemy, and Jinja2 templates. The system generates personalized 7-day workout plans and goal-aligned nutrition or recovery tips from user inputs such as age, weight, fitness goal, and preferred workout intensity.

A key feature of FitBuddy is its feedback loop. Users can submit feedback on an existing plan, and the application uses Gemini to create an updated version while preserving the original plan. The administrative view provides a structured way to review registered users and their plans.

The project demonstrates how generative AI can be integrated into a web application to provide personalized, dynamic fitness support. Its modular architecture also provides a foundation for future improvements such as additional fitness features, richer user management, and expanded deployment options.

Team ID: SWTID-2026-6912